arXiv:2510.01123v1 [cs.LG] 1 Oct 2025

Rethinking Thinking Tokens: LLMs as Improvement Operators

Lovish Madaan[1,2], **Aniket Didolkar**[1,3], **Suchin Gururangan**[4,*], **John Quan**[1], **Ruan Silva**[1], **Ruslan Salakhutdinov**[1], **Manzil Zaheer**, **Sanjeev Arora**[1,5], **Anirudh Goyal**[1]

[1]Meta Superintelligence Labs, [2]University College London, [3]Mila, University of Montreal, [4]Anthropic, [5]Princeton University
[*]Work done at Meta

Reasoning training incentivizes LLMs to produce long chains of thought (long CoT), which among other things, allows them to explore solution strategies with self-checking. This results in higher accuracy, but inflates context length, token/compute cost, and answer latency. We ask: *Can current models leverage their metacognition to provide other combinations on this Pareto frontier, e.g., better accuracy with lower context length and/or latency?* Abstractly, we view the model as an *improvement operator* on its own "thoughts" with a continuum of possible strategies. We study an inference family **Parallel-Distill-Refine (PDR)**, which performs the following: (i) generate diverse drafts in parallel; (ii) *distill* them into a bounded, textual workspace; and (iii) *refine* conditioned on this workspace, producing an output that seeds the next round. Importantly, context length (hence compute cost) is controllable via degree of parallelism, and is no longer conflated with the total number of generated tokens. We report **PDR** instantiations of current models that give better accuracy than long CoT while incurring lower latency. Setting degree of parallelism to 1 yields a subcase **Sequential Refinement (SR)** (iteratively improve a single candidate answer) which provides performance superior to long CoT (at the cost of higher latency). Success of such model orchestrations raises the question whether further training could shift the Pareto frontier. To this end, we train an $8B$ thinking model with Reinforcement Learning (RL) to make it consistent with **PDR** as the inference method. On math tasks with verifiable answers, iterative pipelines surpass single-pass baselines at matched sequential budgets, with **PDR** delivering the largest gains (+11% on AIME 2024 and +9% on AIME 2025).

Correspondence: lovish@meta.com, agi@meta.com

Meta

1 Introduction

Scaling language models to solve harder problems has increasingly relied on eliciting explicit reasoning traces ("thinking tokens") at inference time (Wei et al., 2022; Jaech et al., 2024; Guo et al., 2025). While longer traces often correlate with accuracy, they entangle reasoning depth with sequence length and inherit long-context failure modes (Ghosal et al., 2025). In parallel, the field is gravitating towards *self-improvement*: systems that refine their own outputs via *self-directed operations* (critique, revision, debate, sample-and-select) without expert supervision (Gou et al., 2023; Du et al., 2023b; Irving et al., 2018; Yao et al., 2023; Pan et al., 2025; Zhang et al., 2025).

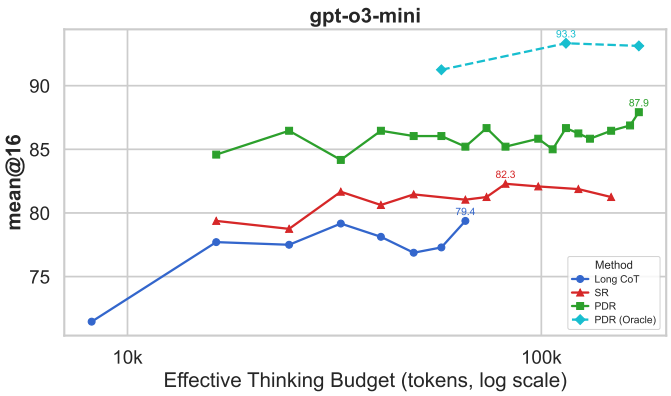

Figure 1 AIME 2024: Accuracy vs. sequential budget B_{seq}. We compare Long CoT, **SR**, and **PDR**; the dashed curve is an oracle upper bound (Oracle-**PDR**) that perfectly transmits any correct solutions under the same budgets to the compact workspace.

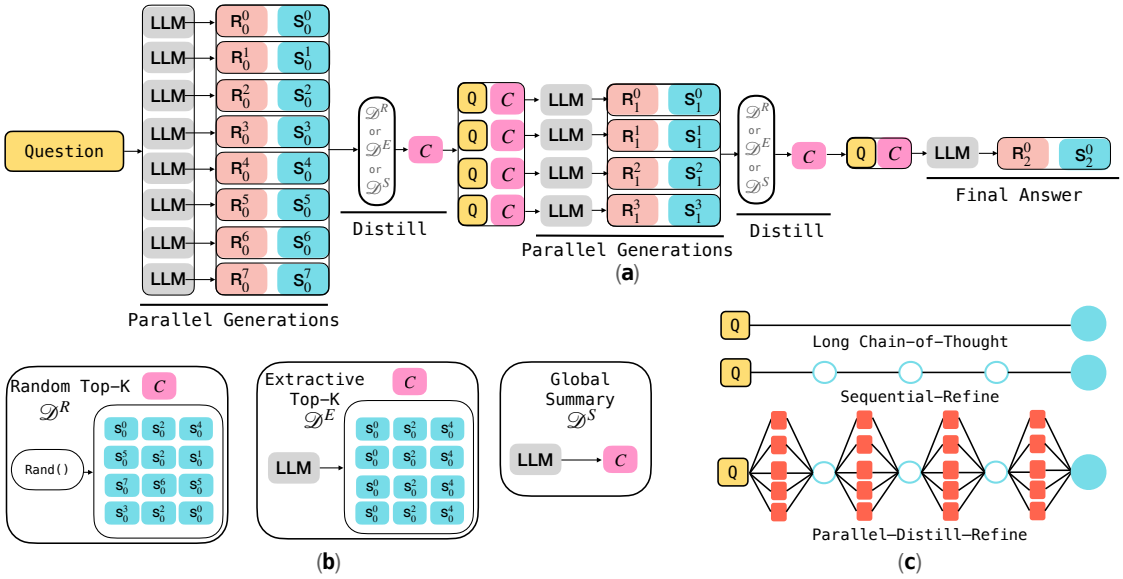

Figure 2 (a) **Parallel-Distill-Refine (PDR)**. In round r, the model generates M_r parallel drafts, then distills them into a compact workspace using one of the schemes in (b); the refined state seeds the next round. (b) Distillation schemes used to build the workspace (e.g., global summary, shared top-k, per-sample top-k, random-k). (c) Three inference regimes. Top-**Long chain-of-thought**: a single, long trace. Middle-**Sequential Refinement (SR)**: one draft updated over short rounds. Bottom-**PDR**: each round spawns M_r drafts, distills into a workspace, and refines. The example shows a 3-round configuration $M = (8, 4, 1)$ (configuration is a hyperparameter, and any other choice is possible). Across panels, the per-call *sequential budget* B_{seq} (latency proxy) is held fixed, while **PDR** increases *total compute* B_{total} via parallelism without increasing per-call context.

Stepping back from the rich body of work, one begins to see LLM inference as a malleable concept; instead of a single "reasoning trace" one encounters choices to be made from a larger pool: generate fresh answers; critique/revise/debate/summarize generated answers; create an updated answer. With this choice comes an unexplored Pareto-frontier:

> *What is the best possible task accuracy achievable after fixing constraints on the inference process, e.g.: (i) total tokens across all generations, (ii) max depth of the generation chain ("latency"), (iii) total context length, and (iv) total compute (which depends on all of the above in complicated and system-dependent ways).*

The confounding factor is that iteration alone does not guarantee progress. Simply asking the model to "try again" risks forgetting useful partial results and repeating earlier mistakes. Naïvely appending all prior attempts to the context recreates long-context failures and scales cost with the number of rounds. Current models suffer from anchoring biases (see Figure 6, 8) as well as forgetfulness. A viable scheme needs a compact state that (i) carries forward salient facts and intermediate results, (ii) flags disagreements and open subgoals, and (iii) remains bounded so each generation (and overall context-size) stays short.

This paper studies inference strategies that generate many tokens with a compact context size. Instead of long chains of thought, inference has phases that generate solutions within the allowed context/token budget and then write a bounded, round-wise summary/report (e.g., listing agreements, contradictions, intermediate results, and open subgoals). The next phase starts with only this summary and uses available workspace for fresh generations (which benefit from accumulated wisdom in the summary). Iterating this process can generate long "thinking" albeit with a bounded context size. [1]

We study two inference instantiations: (i) *Sequential refinement (***SR***)*, where a single artifact (solution, proof,

[1] Such LLMs fall within a traditional framework of *randomized space-bounded computation*. Computational complexity theory Arora & Barak (2007) shows it is capable of surprisingly powerful reasoning, such as determining connectivity of much larger graphs that cannot even fit the working memory; see Section C.

program) is iteratively improved for a fixed number of steps; and (ii) *Parallel-Distill-Refine (***PDR***)* (round-wise workspace), where each round samples M drafts in parallel, distills them into a bounded summary for the next round, and continues. The workspace is not persistent across rounds; it is freshly synthesized for each round. A central challenge is information synthesis: compressing salient facts and intermediate results; flagging uncertainty; and retiring stale information. Its effectiveness hinges on four meta-skills: *verification* (detect and localize errors via self-judging and cross-candidate checks), *refinement* (use feedback/context to improve the artifact), *compression* (retain only past history via bounded summaries rather than replay), and *diversification* (exploratory variation to avoid consensus collapse).

Learning to improve short-context iteration. It is also of interest to teach the model a policy that effectively leverages this improvement operator. Standard RL training for reasoning models typically optimizes a single, long chain-of-thought conditioned on the prompt, with reward on the final answer (Shao et al., 2024; Guo et al., 2025). **PDR**, by contrast, comprises multiple short iterations that read a bounded summary, write a refinement, and re-synthesize a fresh summary. This creates a train-test mismatch in the information flow (short updates vs. one long trace). To make sure training is consistent with deployment, we optimize an objective that unrolls the operator itself during training: sample M short drafts, distill them into a compact summary, and condition on the prompt plus that summary to produce a refined attempt. We use verifiable rewards to supervise the end-to-end computation. This objective narrows the train–test gap.

Results. On math tasks, iterative pipelines surpass single-pass baselines at matched sequential budgets; with shallow **PDR** delivering the largest gains (e.g., +11% on AIME 2024 and +9% on AIME 2025). Making training consistent with inference, via an operator-consistent RL objective that optimizes the same read/write interface used at test time yields further improvements (e.g., $\sim 5\%$ on AIME 2024 and AIME 2025 when mixing standard and operator RL). These findings suggest that iteration with short contexts and compact summaries can substitute for long traces while holding latency fixed.

2 Background & Related Work

Overview. This section situates our work among several threads that seek to scale test-time reasoning: long-trace chains of thought and self-consistency; self-improvement and debate; structured search (trees/graphs of thoughts); multi-trace selection and aggregation; turning compute into supervision; memory/summarization; adaptive multi-turn RL; and learning-to-search/planning. We view these through a unified lens: *inference as a round-wise improvement operator under explicit budgets*; holding the per-call *sequential* budget $\mathrm{B}_{\mathrm{seq}}$ fixed while varying *total* compute $\mathrm{B}_{\mathrm{total}}$. We do not claim the micro-primitives themselves are new: parallel sampling, aggregation and selection (Wang et al., 2023; Fu et al., 2025; Zhao et al., 2025), sequential revision, critique-revise-verify and debate (Madaan et al., 2023; Gou et al., 2023; Shinn et al., 2023; Du et al., 2023b), structured search (Yao et al., 2023; Besta et al., 2024), and summarization/memory (Wu et al., 2025; Yang et al., 2024) are all well explored. Our contribution is to (i) formalize these pieces within a single round-wise operator (**SR**, **PDR**), (ii) analyze compute-normalized depth via shallow parallel rounds and distillation at matched B_{seq} while varying B_{total}, and (iii) make training consistent with deployment via *operator-consistent RL*. The text below organizes prior work along these axes and clarifies similarities and differences.

Test-time reasoning with long traces. Eliciting step-by-step "chains of thought" improves accuracy on multi-step tasks (Wei et al., 2022). Recent "reasoning" models (e.g., OpenAI *o1*, Deepseek *r1*) explicitly trade more test-time thinking for better results, increasing tokens and latency (Jaech et al., 2024; Guo et al., 2025). Sampling multiple traces and aggregating answers (self-consistency) further boosts performance but scales cost with the number of samples (Wang et al., 2023). Our approach targets a complementary design point: keep each call short while letting evidence accumulate across rounds via a bounded, re-synthesized summary.

Self-improvement. A growing line of work lets models critique and refine their own outputs: *Self-Refine* alternates self-feedback and revision (Madaan et al., 2023); *Reflexion* maintains textual memory to guide subsequent attempts (Shinn et al., 2023); *CRITIC* verifies with tools and revises accordingly (Gou et al., 2023). Multi-agent debate improves factuality and reasoning via argumentation and adversarial checking (Irving et al., 2018; Du et al., 2023b). Our operator shares the self-improvement spirit but constrains per-call context by using a round-wise, re-synthesized compact state $C^{(r)}$ instead of replaying full histories. *Compute as Teacher* (CaT) synthesizes a single reference from parallel rollouts and optimizes the policy toward it, converting extra